

Presented by Ayush Indapure

Student ID : 224880003

SOFTWARE QUALITY AND TESTING

BOUNDARY VALUE ANALYSIS & EQUIVALENCE CLASS TESTING

This week I learned about two important testing strategies

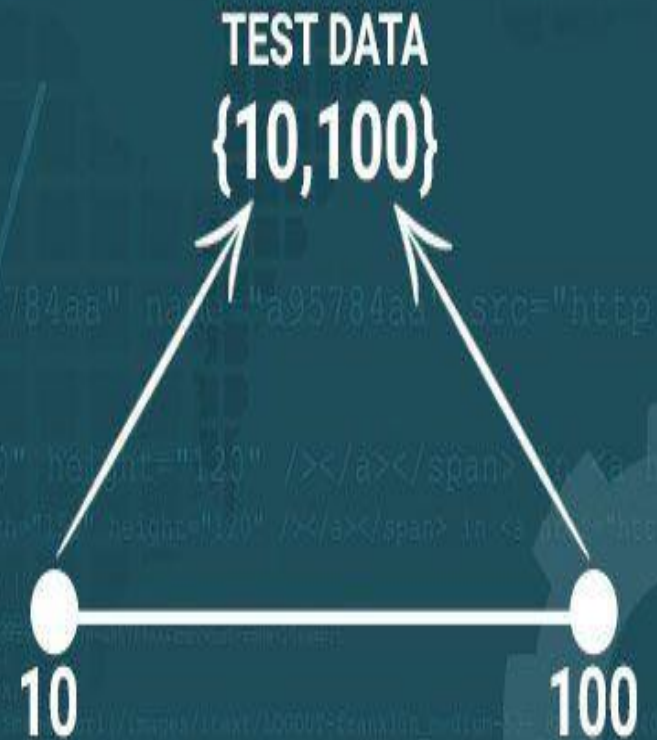
- Boundary Value Analysis (BVA)
- Equivalence Class Testing (ECT)

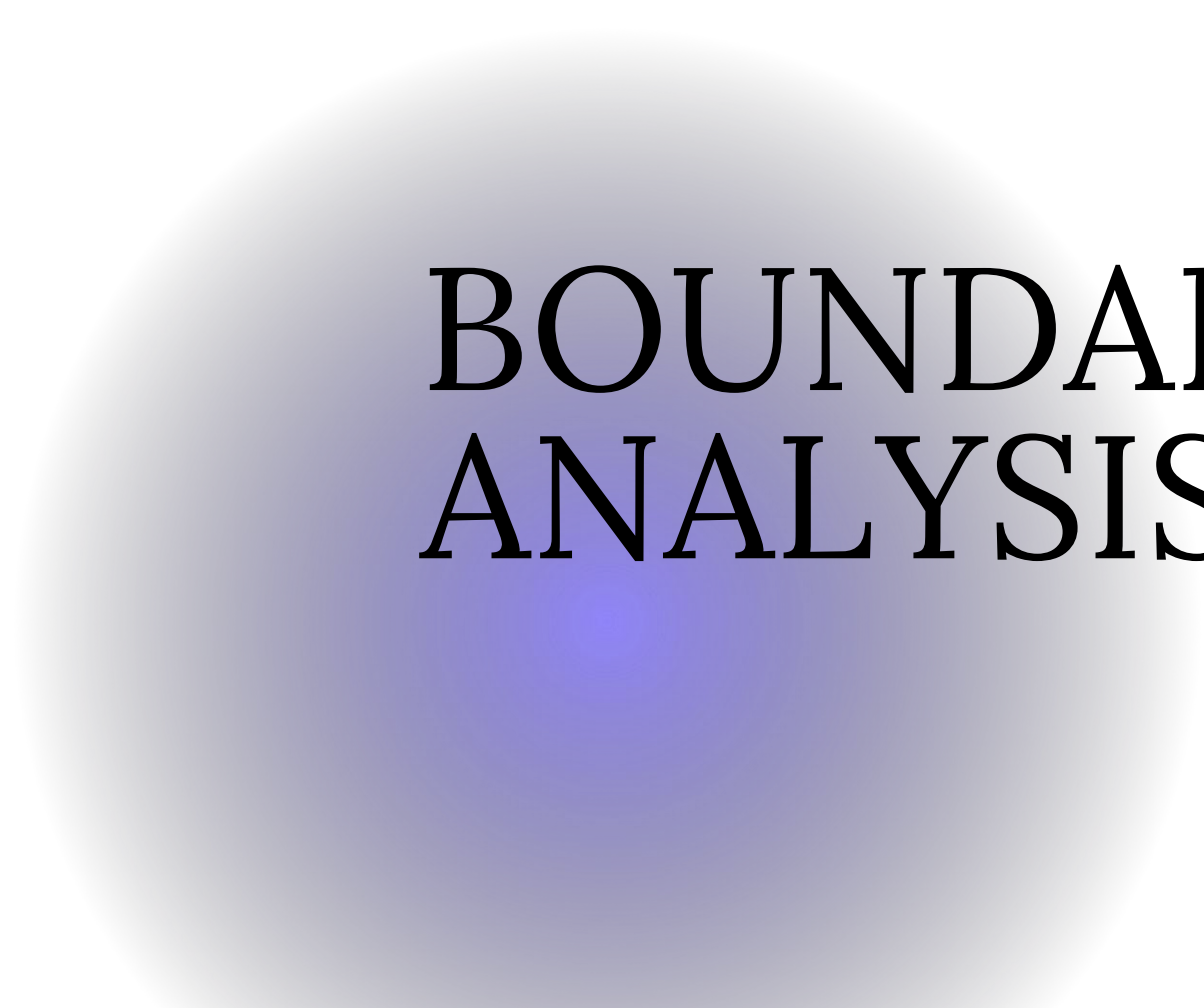
Both techniques help testers design **efficient test cases** without testing every possible input.

Instead of testing thousands of inputs, we focus on **critical input ranges and edge cases**.

These strategies are commonly used in real-world software systems where inputs have **specific limits or constraints**.

Boundary Value Analysis





BOUNDARY VALUE ANALYSIS

BVA

(Boundary Value Analysis)

**BOUNDARY VALUE
ANALYSIS FOCUSES
ON TESTING VALUES
AT THE EDGES OF
INPUT RANGES.**

BOUNDARY VALUE ANALYSIS

Boundary Value Analysis (BVA) is a software testing technique that basically focuses on testing input values at the edges of defined ranges, as defects are most likely to occur at these boundaries rather than in the middle. .

This black-box testing method is based on the observation that errors frequently happen when values are just below, at, or just above the valid limits of an equivalence partition. Testers design test cases to cover the minimum, just above minimum, nominal, just below maximum, and maximum values of a range, often testing both valid and invalid partitions.

Black Box Testing Techniques

2. Boundary Value Analysis



REAL EXAMPLE I'VE
SEEN (DSA CONTEXT)

EXAMPLE THROUGH DATA STRUCTURES AND ALGORITHMS

While practicing **Data Structures and Algorithms**, I noticed boundary problems often appear in array indexing.

Example:

```
int[] arr = new int[10];
```

Valid indexes are:

0 – 9

If a program tries to access:

```
arr[10]
```

it causes an **ArrayIndexOutOfBoundsException**.

So the boundary values are:

0, 1, 8, 9

Testing these helps ensure the algorithm handles edge cases correctly.

This is very similar to Boundary Value Analysis used in software testing.

EQUIVALENCE CLASS TESTING(ECT)

Equivalence Class Testing divides input data into **groups (classes)** where the program should behave the same. Instead of testing every possible input, we select **one representative value from each class**.

This black-box testing method is based on the observation that errors frequently happen when values are just below, at, or just above the valid limits of an equivalence partition. Testers design test cases to cover the minimum, just above minimum, nominal, just below maximum, and maximum values of a range, often testing both valid and invalid partitions.

Black Box Testing Techniques

1. Equivalence Class Testing



EXAMPLE THROUGH DATA STRUCTURES AND ALGORITHMS

A common real-world example is **password validation**.

Suppose a system requires:

Password length: 8 – 16 characters

Equivalence classes would be:

Valid class

8–16 characters

Invalid classes

< 8 characters

> 16 characters

Instead of testing every possible length, testers choose **representative values** like:

7, 8, 12, 16, 17

HOW THESE METHODS ARE USED IN REAL SOFTWARE

Both testing techniques are widely used in real systems:

Examples include:

- Banking systems (transaction limits)
 - E-commerce websites (quantity limits)
 - Input validation in web forms
 - Date calculators and scheduling systems
 - API request parameter validation

By focusing on boundaries and input groups, testers can **reduce the number of tests while still achieving high coverage.**

REFLECTION

Both testing techniques are widely used in real systems:

Examples include:

- Banking systems (transaction limits)
 - E-commerce websites (quantity limits)
 - Input validation in web forms
 - Date calculators and scheduling systems
 - API request parameter validation

By focusing on boundaries and input groups, testers can **reduce the number of tests while still achieving high coverage.**

EVIDENCE OF LEARNING ACTIVITY

During the active learning session I explored:

- Boundary testing for a **date calculator program**
 - Using JUnit test cases to validate date boundaries
 - Testing edge cases such as:
 - Start of month
 - End of month
 - Leap year conditions

The screenshot displays a web browser window with the URL `d2l.deakin.edu.au/d2l/home/1777008`. The page features a navigation bar with various icons and a main content area. At the top, there is a banner with a landscape image and text encouraging the use of Studiosity+ for support. Below this, a section titled "Week 3 - Boundary Testing & Equivalence Testing" is highlighted, with a "Next >" link. A progress bar indicates "100% completed - 8 of 8 resources viewed". A list of resources follows, each with a "Seen" status and a green checkmark:

- Value Testing
- Class Testing
- JUnit and randomised data
- JUnit code generation
- JUnit Recording
- JUnit using JUnit
- JUnit

At the bottom of the list, there is a "See all Resources >" link. To the right of the main content, there is a sidebar with a "Unit Staff" section listing Dr Zubair Baig as the Unit Chair, with contact information for Geelong Waurn Ponds Campus. Below this, there is a "Studiosity" section with a "Need help with assessment? Access trusted 24/7 feedback" message and a "Go to my account" button. At the bottom of the sidebar, there is a "This Cloud Timetabling App" section with a brief description and a link to the app.

Thank You!